

Rangkuman Algorithm Design & Analysis

Nama : Wilbert Chandra

NIM : 2602113666

Kelas : LB09

Jurusan : Data Science

Sesi 1 : Peran Algoritma dalam Pemrograman

Definisi Algoritma

- Definisi: Urutan langkah komputasi yang jelas dan terdefinisi dengan baik yang mengubah nilai input menjadi nilai output, digunakan untuk memecahkan masalah komputasi.
- Contoh Pengurutan: Mengurutkan angka dalam urutan meningkat menggunakan berbagai teknik dan alat.
- Algoritma yang Benar: Berhenti dan menghasilkan solusi yang benar untuk setiap instance masalah.

Masalah yang Diselesaikan oleh Algoritma

- Proyek Genom Manusia
- Manajemen data internet
- Alokasi sumber daya di perusahaan komersial
- Pemrograman linier untuk solusi optimal di berbagai industri

Pseudocode

- Spesifikasi algoritma tingkat tinggi menggunakan campuran bahasa Inggris dan notasi matematika.
- Komponen termasuk variabel, loop, percabangan, dan fungsi modular.

Algoritma sebagai Teknologi

- Efisiensi: Pentingnya memilih algoritma yang efisien untuk mengoptimalkan sumber daya komputasi (waktu dan memori).
- Contoh: Perbandingan antara Insertion Sort dan Merge Sort, menggambarkan pentingnya efisiensi algoritma dibandingkan kekuatan komputasi mentah.

Ringkasan

- Algoritma mengubah input menjadi output melalui langkah-langkah komputasi.
- Algoritma memecahkan berbagai masalah praktis di berbagai bidang.

- Algoritma yang efisien sangat penting untuk kinerja sistem, sebanding dengan memilih perangkat keras yang cepat.
- Pengetahuan algoritma yang kuat mendefinisikan seorang programmer yang terampil.

Sesi 2 : Induksi Matematis dan Fungsi Rekursif

Pendekatan Metodis untuk Mengatasi Masalah Rekursif:

- Mengidentifikasi Kasus Dasar: Kasus paling sederhana yang bisa dipecahkan secara langsung.
- Mengidentifikasi Kasus Rekursif: Kasus di mana fungsi memanggil dirinya sendiri dengan input yang lebih sederhana atau lebih kecil.
- Menulis Tanda Tangan Fungsi: Mendefinisikan fungsi dan parameternya.
- Menggabungkan Kasus Dasar dan Rekursif: Memastikan fungsi menangani keduanya.
- Menguji Fungsi: Validasi dengan input yang berbeda untuk memastikan akurasi.

Pentingnya Algoritma

- Algoritma penting karena mengoptimalkan penggunaan sumber daya, meningkatkan efisiensi, dan sangat penting dalam pengembangan sistem perangkat lunak dan perangkat keras.

Jenis Algoritma

- Algoritma dapat dikategorikan berdasarkan fungsinya, seperti algoritma pengurutan, algoritma pencarian, dan algoritma graf.

Efisiensi Algoritma

- Efisiensi algoritma diukur dengan kompleksitas waktu dan ruangnya. Algoritma yang efisien meminimalkan penggunaan sumber daya dan meningkatkan kinerja.

Merge Sort vs. Insertion Sort

- Contoh praktis yang menunjukkan bahwa bahkan pada perangkat keras yang lebih lambat, merge sort dapat mengungguli insertion sort saat menangani input besar karena efisiensinya yang lebih baik.

Algoritma dalam Teknologi Modern

- Algoritma memainkan peran penting dalam berbagai teknologi, termasuk arsitektur komputer canggih, antarmuka pengguna grafis (GUI), jaringan, dan pembelajaran mesin. Algoritma yang efisien berkontribusi signifikan terhadap kinerja sistem secara keseluruhan.

Aplikasi Algoritma

- Algoritma diterapkan di banyak bidang seperti bioinformatika, mesin pencari internet, dan manufaktur. Kemampuan mereka untuk memecahkan masalah kompleks membuat mereka sangat diperlukan di banyak area.

Sesi 3: Struktur Data

Pendahuluan

Struktur data adalah cara untuk mengatur dan menyimpan data agar memungkinkan akses dan modifikasi yang efisien. Mereka sangat penting untuk merancang algoritma yang efisien dan mengelola dataset yang besar.

Struktur Data Dasar

1. **Array**: Koleksi elemen dengan ukuran tetap dan tipe yang sama.
2. **Linked List**: Koleksi elemen di mana setiap elemen menunjuk ke elemen berikutnya.
3. **Stack**: Struktur data LIFO (Last In, First Out).
4. **Queue**: Struktur data FIFO (First In, First Out).

Struktur Data Lanjutan

1. **Trees**: Struktur hierarkis dengan node yang terhubung oleh tepi.
 - **Binary Trees**: Setiap node memiliki paling banyak dua anak.
 - **Binary Search Trees**: Binary trees dengan node yang terurut.
 - **Heaps**: Pohon khusus untuk operasi antrean prioritas.
2. **Graphs**: Koleksi node yang terhubung oleh tepi, mewakili hubungan.
 - **Directed and Undirected Graphs**: Tergantung pada arah tepi.

Sesi 4: Menganalisis Waktu Eksekusi Algoritma

Fungsi dan Notasi Matematika

- **Monotonicity**: Fungsi yang seluruhnya non-meningkat atau non-menurun.
- **Polynomials**: Karakteristik dan perilaku fungsi polinomial.
- **Exponentials**: Pertumbuhan dan peluruhan eksponensial.
- **Logarithms**: Properti dan aplikasi fungsi logaritmik.
- **Factorials**: Definisi dan signifikansi dalam analisis kompleksitas.
- **Functional Iteration**: Metode iteratif untuk fungsi.
- **Iterated Logarithm Function**: Fungsi yang tumbuh lambat digunakan dalam analisis kompleksitas.
- **Fibonacci Numbers**: Penjelasan dan hubungannya dengan rasio emas.

Notasi Asimtotik

- **O-notation**: Untuk menggambarkan batas atas suatu fungsi.
- **Ω -notation**: Untuk menggambarkan batas bawah.

- **Θ -notation:** Untuk menggambarkan batas ketat, menunjukkan batas atas dan bawah.

Sesi 5: Analisis Struktur Data: Stack, Queue, Pohon, dan Binary Tree

Tipe Data Abstrak (ADT)

- **Definisi:** Model untuk struktur data yang menyediakan serangkaian operasi tanpa menentukan implementasinya.
- **Contoh:** Stack, Queue, Tree, dan Graph.

Struktur Data Stack

- **Operasi:** PUSH (menambahkan elemen), POP (menghapus elemen teratas), EMPTY (memeriksa apakah stack kosong).
- **Implementasi:** Dapat diimplementasikan menggunakan array atau linked list.
- **Prinsip LIFO:** Aturan Last In, First Out untuk operasi stack.

Struktur Data Queue

- **Operasi:** ENQUEUE (menambahkan elemen), DEQUEUE (menghapus elemen depan), EMPTY (memeriksa apakah queue kosong).
- **Implementasi:** Dapat diimplementasikan menggunakan array atau linked list.
- **Prinsip FIFO:** Aturan First In, First Out untuk operasi queue.

Struktur Data Pohon

- **Definisi:** Struktur data hierarkis dengan node yang terhubung oleh tepi.
- **Karakteristik:** Node memiliki hubungan parent-child.
- **Binary Tree:** Setiap node memiliki paling banyak dua anak, kiri dan kanan.

Traversal Pohon

- **Jenis Traversal:** Pre-order (parent-kiri-kanan), In-order (kiri-parent-kanan), Post-order (kiri-kanan-parent).

Sesi 6: Analisis Struktur Data: Graph (Pencarian Dasar & Traversal), Priority Queue, dan Heap

Struktur Data Graph

- **Definisi:** Graph adalah ADT yang terdiri dari node (vertex) dan tepi.
- **Jenis Graph:** Graph berarah (digraph) dan graph tak berarah.
- **Derajat Node:** Derajat masuk dan derajat keluar node.

Representasi Graph

- **Adjacency Matrix:** Mewakili kedekatan antar node.
- **Adjacency List:** Representasi daftar dari koneksi graph.
- **Cost Matrix:** Mewakili biaya atau jarak antara node.

Traversal Pohon dalam Graph

- **Depth First Search (DFS):** Menelusuri sedalam mungkin sebelum mundur.
- **Breadth First Search (BFS):** Menjelajahi semua node pada kedalaman saat ini sebelum melanjutkan ke node pada tingkat kedalaman berikutnya.

Priority Queue

- **Definisi:** Struktur data di mana setiap elemen memiliki prioritas.
- **Operasi:** Penyisipan, menemukan minimum, penghapusan.
- **Contoh:** Aplikasi dunia nyata seperti penjadwalan pekerjaan.

Struktur Data Heap

- **Definisi:** Struktur berbasis pohon khusus yang memenuhi properti heap.
- **Operasi:** Penyisipan, penghapusan, menemukan minimum.
- **Heapsort:** Algoritma penyortiran efisien menggunakan heap.

Sesi 7: Divide and Conquer

Pendahuluan ke Algoritma Strassen

- **Motivasi:** Perkalian matriks lebih cepat dibandingkan metode tradisional.
- **Peningkatan:** Mengurangi jumlah perkalian dari delapan menjadi tujuh.

Langkah-langkah Algoritma Strassen

- **Langkah 1:** Mirip dengan prosedur MATRIX-MULTIPLY-RECURSIVE.
- **Langkah 2:** Pembuatan 10 matriks dari submatriks A dan B.
- **Langkah 3:** Perkalian rekursif matriks $n/2 \times n/2$ sebanyak 7 kali.
- **Langkah 4:** Memperbarui matriks hasil C dengan menambahkan/mengurangkan matriks yang dihitung.

Relasi Rekurens

- **Analisis Kompleksitas:** Penurunan relasi rekurens dan dampaknya pada kompleksitas waktu.

Ringkasan

- **Efisiensi:** Algoritma Strassen menawarkan solusi yang lebih efisien untuk perkalian matriks dengan memanfaatkan trik aljabar untuk mengurangi jumlah operasi yang diperlukan.

Sesi 8: Divide and Conquer: Rekurens

Pendahuluan ke Perkalian Matriks

- **Pendekatan Divide and Conquer:** Penjelasan pemecahan perkalian matriks menggunakan divide and conquer.
- **Algoritma yang Lebih Cepat:** Diskusi tentang algoritma canggih seperti Strassen untuk efisiensi yang lebih baik.

Konsep Matematis Kunci

- **Functional Iteration:** Menggunakan metode iteratif untuk menerapkan fungsi.
- **Iterated Logarithm Function:** Fungsi yang tumbuh lambat digunakan dalam analisis kompleksitas.
- **Fibonacci Numbers:** Penjelasan tentang properti mereka dan hubungannya dengan rasio emas.

Notasi Asimtotik

- **O-notation, Ω -notation, Θ -notation:** Diskusi rinci tentang notasi yang berbeda untuk mengkarakterisasi fungsi dan pentingnya dalam analisis algoritma.

Sesi 9: Algoritma Randomized: Hiring Problem

Pendahuluan ke Struktur Data

- **Definisi:** Struktur data adalah cara untuk mengatur dan menyimpan data agar memungkinkan akses dan modifikasi yang efisien.
- **Pentingnya:** Penting untuk merancang algoritma yang efisien dan mengelola dataset yang besar.

Struktur Data Dasar

- **Array:** Koleksi elemen dengan ukuran tetap dan tipe yang sama.
- **Linked Lists:** Koleksi elemen di mana setiap elemen menunjuk ke elemen berikutnya.
- **Stacks:** Struktur data LIFO (Last In, First Out).
- **Queues:** Struktur data FIFO (First In, First Out).

Struktur Data Lanjutan

- **Trees:** Struktur hierarkis dengan node yang terhubung oleh tepi.
 - **Binary Trees:** Setiap node memiliki paling banyak dua anak.

- **Binary Search Trees:** Binary trees dengan node yang terurut.
- **Heaps:** Pohon khusus untuk operasi antrean prioritas.
- **Graphs:** Koleksi node yang terhubung oleh tepi, mewakili hubungan.
 - **Directed and Undirected Graphs:** Tergantung pada arah tepi.

Session 10: Greedy Method

1. Greedy Algorithms

- **Introduction:** Algoritma greedy membuat pilihan optimal secara lokal di setiap langkah dengan harapan menemukan optimum global.
- **Konsep Utama:**
 - **Greedy Choice Property:** Optimum global dapat dicapai dengan memilih optimum lokal.
 - **Optimal Substructure:** Solusi optimal untuk masalah mengandung solusi optimal untuk submasalahnya.

2. Contoh dan Aplikasi

- **Masalah Seleksi Aktivitas:** Memilih jumlah maksimum aktivitas yang tidak tumpang tindih.
- **Masalah Knapsack Fraksional:** Memaksimalkan total nilai barang yang bisa dibawa dalam knapsack.

3. Pengkodean Huffman

- **Pengenalan Pengkodean Huffman:** Metode untuk kompresi data tanpa kehilangan.
- **Membangun Pohon Huffman:** Menggunakan antrian prioritas untuk membuat pohon biner optimal untuk mengkodekan data.
- **Pengkodean dan Dekode:** Menetapkan kode biner untuk karakter berdasarkan frekuensinya.

Session 11: Greedy Methods: Activity-selection Problem

1. Pemrograman Dinamis

- **Pengenalan:** Pemrograman dinamis adalah metode untuk menyelesaikan masalah kompleks dengan memecahnya menjadi submasalah yang lebih sederhana.
- **Prinsip:**
 - **Optimal Substructure:** Masalah memiliki struktur optimal jika solusi optimal dapat dibangun dari solusi optimal submasalahnya.
 - **Overlapping Subproblems:** Masalah dapat dipecah menjadi submasalah yang digunakan berulang kali.

2. Contoh dan Aplikasi

- **Deret Fibonacci:** Menggunakan pemrograman dinamis untuk menghitung angka Fibonacci dengan efisien.
- **Longest Common Subsequence (LCS):** Menemukan subsekuensi terpanjang yang umum dari dua urutan, dengan aplikasi di bioinformatika dan perbandingan teks.

3. Teknik

- **Memoization:** Menyimpan hasil dari panggilan fungsi yang mahal dan menggunakannya kembali ketika input yang sama muncul kembali.

Session 12: Huffman Coding

1. Pengkodean Huffman

- **Pengenalan Pengkodean Huffman:** Metode untuk kompresi data tanpa kehilangan.
- **Membangun Pohon Huffman:** Menggunakan antrian prioritas untuk membuat pohon biner optimal untuk mengkodekan data.
- **Pengkodean dan Dekode:** Menetapkan kode biner untuk karakter berdasarkan frekuensinya.

2. Contoh Pengkodean dan Dekode

- **Contoh Praktis:** Menunjukkan contoh pengkodean dan dekode menggunakan kode Huffman.

Session 13: Amortized Analysis

1. Tabel Dinamis

- **Definisi:** Tabel dinamis menyesuaikan ukuran saat jumlah item berubah, memungkinkan penggunaan memori yang efisien.
- **Operasi:**
 - **TABLE-INSERT:** Menambahkan item ke tabel.
 - **TABLE-DELETE:** Menghapus item dari tabel.
- **Tujuan:** Mempertahankan biaya amortasi $O(1)$ untuk penyisipan dan penghapusan sambil memastikan penggunaan ruang yang efisien.

2. Analisis Amortisasi

- **Gambaran Umum:** Analisis amortisasi memberikan cara untuk meratakan biaya operasi selama rangkaian, memastikan bahwa operasi yang mahal diimbangi oleh yang lebih murah.
- **Metode:**
 - **Aggregate Analysis:** Menghitung batas atas total biaya rangkaian operasi.
 - **Accounting Method:** Menetapkan biaya amortisasi berbeda untuk operasi untuk menyederhanakan analisis.
 - **Potential Method:** Menggunakan fungsi potensial untuk merepresentasikan "energi tersimpan" dari struktur data, yang dapat digunakan untuk mengimbangi biaya di masa depan.

3. Potential Method Diterapkan pada Peningkatan Penghitung Biner

- **Skenario:** Mengevaluasi biaya peningkatan penghitung biner.
- **Perhitungan Biaya Amortisasi:** Menunjukkan bahwa biaya terburuk dari n operasi peningkatan adalah $O(n)$ dengan menyebarkan biaya flipping bit selama beberapa operasi.

Session 14: Review I

1. Divide-and-Conquer

- **Pengenalan Divide-and-Conquer:**
 - **Definisi dan Ide Dasar:** Algoritma divide-and-conquer memecah masalah menjadi submasalah yang lebih kecil, menyelesaikannya secara independen, dan menggabungkan hasilnya.
 - **Binary Search:**
 - Penjelasan dan implementasi binary search.
 - Analisis kompleksitas waktu dan perbandingan dengan linear search.
 - **Merge-Sort:**
 - Proses langkah-demi-langkah algoritma merge-sort.
 - Analisis kompleksitas waktu dan keuntungan dibandingkan algoritma pengurutan lainnya.
 - **Multiplikasi Matriks:**
 - Metode rekursif untuk multiplikasi matriks.
 - Algoritma Strassen untuk multiplikasi matriks yang lebih efisien.
- 2. **Randomized Algorithms**
 - **Pengenalan Algoritma Random:**
 - **Definisi dan Contoh:** Algoritma random.
 - **Manfaat Menggunakan Randomness:** Keuntungan menggunakan random dalam desain algoritma.
 - **Randomized Linear Search:**
 - Implementasi dan analisis pencarian linear random.
 - Perbandingan dengan pencarian linear deterministik.

Session 15: Pemotongan Batang (Rod Cutting)

1. Pemotongan Batang (Rod Cutting)

Definisi: Pemotongan batang adalah masalah klasik dalam pemrograman dinamis di mana kita mencari cara memotong batang panjang menjadi beberapa bagian untuk memaksimalkan keuntungan.

Pendekatan Masalah:

- Misalkan kita memiliki batang panjang n dan daftar harga untuk setiap panjang tertentu.
- Tujuan kita adalah menentukan cara memotong batang tersebut sehingga total nilai potongan-potongan maksimal.

Contoh:

- Batang panjang 4 dengan daftar harga: [1, 5, 8, 9].
- Pemotongan terbaik adalah [2, 2] dengan nilai total 10.
- 2. **Pemrograman Dinamis untuk Pemotongan Batang:**
 - **Pendekatan Bottom-Up:**

- Gunakan tabel untuk menyimpan nilai maksimum untuk setiap panjang batang.
- Mulai dari panjang 1 hingga n , dan untuk setiap panjang, tentukan apakah akan dipotong atau tidak untuk mencapai nilai maksimum.
- **Kompleksitas Waktu:** Pendekatan ini memiliki kompleksitas waktu $O(n^2)$, yang lebih efisien dibandingkan pendekatan brute-force.

Sesi 16: Pemrograman Dinamis - Perkalian Rantai Matriks

1. Pengantar Pembelajaran Mesin

- **Definisi:** Pembelajaran mesin adalah subset dari kecerdasan buatan yang melibatkan pelatihan algoritma untuk belajar dari data dan membuat prediksi atau keputusan.
- **Jenis Pembelajaran Mesin:**
 - **Pembelajaran Terawasi:** Pelatihan dengan data berlabel.
 - **Pembelajaran Tak Terawasi:** Pelatihan dengan data tidak berlabel.
 - **Pembelajaran Penguatan:** Pembelajaran melalui interaksi dengan lingkungan.

2. Algoritma dan Model Utama

- **Pembelajaran Terawasi:**
 - **Regresi:** Regresi linear, regresi logistik.
 - **Klasifikasi:** Pohon keputusan, mesin vektor pendukung, k-nearest neighbors.
- **Pembelajaran Tak Terawasi:**
 - **Pengelompokan:** Pengelompokan k-means, pengelompokan hierarkis.
 - **Pengurangan Dimensi:** Analisis komponen utama (PCA), t-SNE.

3. Evaluasi dan Validasi Model

- **Metrik:** Akurasi, presisi, recall, skor F1, kurva ROC.
- **Teknik:** Cross-validation, train-test split, bootstrapping.

4. Aplikasi Praktis

- **Kasus Penggunaan Industri:** Deteksi penipuan, sistem rekomendasi, pemeliharaan prediktif, segmentasi pelanggan.

5. Tantangan dan Pertimbangan

- **Overfitting dan Underfitting:** Menyeimbangkan kompleksitas model.
- **Kualitas Data:** Menangani nilai hilang, outlier, dan set data yang tidak seimbang.
- **Etika dan Bias:** Memastikan keadilan dan transparansi dalam sistem AI.

Sesi 17: Pemrograman Dinamis - Suburutan Umum Terpanjang

1. Analisis Amortisasi

- **Pengantar Analisis Amortisasi:**
 - **Definisi dan Tujuan:** Analisis amortisasi digunakan untuk mengukur biaya rata-rata operasi selama sejumlah operasi.
 - **Metode Agregat:** Menghitung batas atas total biaya serangkaian operasi.
 - **Metode Akuntansi:** Menetapkan biaya amortisasi berbeda untuk operasi yang berbeda untuk menyederhanakan analisis.
 - **Metode Potensial:** Menggunakan fungsi potensial untuk mewakili "energi yang tersimpan" dari struktur data, yang dapat digunakan untuk mengimbangi biaya di masa mendatang.

2. Struktur Data Lanjutan

- **Heap Fibonacci:**
 - **Definisi dan Sifat:** Heap Fibonacci adalah struktur data yang mendukung operasi heap dengan biaya amortisasi yang lebih rendah.
 - **Operasi dan Analisis Amortisasi:** Melakukan operasi seperti penyisipan, penghapusan minimum, dan penggabungan dengan biaya amortisasi yang efisien.
- **Pohon Splay:**
 - **Pengantar dan Sifat:** Pohon splay adalah pohon biner pencarian yang melakukan rotasi untuk menjaga elemen yang sering diakses lebih dekat ke akar.

Sesi 18: Teori Graf - Komponen Terhubung Kuat

1. Pengantar Teori Graf

- **Definisi dan Istilah Dasar:** Graf adalah struktur yang terdiri dari simpul dan sisi yang menghubungkan simpul-simpul tersebut.
- **Penerapan Teori Graf:** Banyak digunakan dalam jaringan komputer, jalur transportasi, dan algoritma pencarian jalur.

2. Algoritma DFS dan BFS

- **Depth-First Search (DFS):** Menelusuri graf dengan menjelajahi sepanjang cabang sebelum kembali ke simpul sebelumnya.
- **Breadth-First Search (BFS):** Menelusuri graf dengan menjelajahi semua simpul pada satu tingkat kedalaman sebelum beralih ke tingkat berikutnya.

Sesi 19: Teori Graf - Jalur Terpendek

1. Pengantar Algoritma Jalur Terpendek

- **Algoritma Dijkstra:** Menemukan jalur terpendek dari satu sumber ke semua simpul lain dalam graf berberat non-negatif.
- **Algoritma Bellman-Ford:** Menangani graf dengan bobot negatif dan mendeteksi siklus dengan bobot negatif.

Sesi 20: Algoritma Greedy - Pengenalan

1. Pengantar Algoritma Greedy

- **Definisi:** Algoritma greedy membuat pilihan lokal optimal pada setiap langkah dengan harapan menghasilkan solusi global optimal.
- **Contoh Masalah Greedy:** Pencarian pohon rentang minimum, penjadwalan tugas, dan masalah koin perubahan.

2. Algoritma Prim dan Kruskal

- **Algoritma Prim:** Membangun pohon rentang minimum dengan menambahkan sisi terpendek yang menghubungkan simpul baru ke pohon yang ada.
- **Algoritma Kruskal:** Membangun pohon rentang minimum dengan menambahkan sisi terpendek yang tidak membentuk siklus, menggunakan teknik penyatuan dan temukan (union-find).

Sesi 21: Algoritma Greedy - Penyandian Huffman

1. Pengantar Penyandian Huffman

- **Definisi dan Penggunaan:** Penyandian Huffman adalah teknik kompresi lossless yang digunakan untuk mengurangi ukuran data.
- **Langkah-Langkah Penyandian Huffman:** Membangun pohon Huffman berdasarkan frekuensi kemunculan simbol dan menghasilkan kode biner untuk setiap simbol.

2. Contoh dan Aplikasi

- **Contoh Penyandian dan Dekoding:** Menunjukkan langkah-langkah penyandian dan dekoding menggunakan kode Huffman.

Sesi 22: Peninjauan I - Algoritma Greedy dan Dinamis

1. Peninjauan Algoritma Greedy

- **Kelebihan dan Kekurangan:** Diskusi tentang kapan algoritma greedy bekerja dengan baik dan kapan tidak.
- **Studi Kasus:** Contoh-contoh nyata di mana algoritma greedy berhasil diterapkan.

2. Peninjauan Pemrograman Dinamis

- **Kelebihan dan Kekurangan:** Diskusi tentang keefektifan dan keterbatasan pendekatan pemrograman dinamis.
- **Studi Kasus:** Contoh-contoh nyata di mana pemrograman dinamis berhasil diterapkan.

Sesi 23: Algoritma Pendekatan - Vertex Cover

1. Pengantar Topik Lanjutan

- **Ikhtisar Konsep Lanjutan:** Penjelasan tentang pentingnya mengikuti perkembangan teknologi terbaru.
- **Studi Kasus:** Analisis mendalam tentang penerapan konsep lanjutan dalam berbagai industri.

2. Tantangan dan Solusi

- **Identifikasi Tantangan Umum:** Diskusi tentang tantangan umum yang dihadapi saat menerapkan teknologi lanjutan.
- **Solusi Praktis:** Strategi dan solusi untuk mengatasi tantangan tersebut.

Sesi 24: Algoritma Pendekatan - Traveling Salesperson

1. Pengantar Algoritma Pendekatan

- **Definisi dan Ruang Lingkup:** Algoritma pendekatan digunakan untuk menemukan solusi mendekati optimal untuk masalah NP-hard dalam waktu yang wajar.
- **Kebutuhan Pendekatan:** Karena kesulitan komputasi untuk menemukan solusi tepat, algoritma pendekatan menyediakan alternatif praktis.

2. Masalah Subset-Sum

- **Pernyataan Masalah:** Diberikan satu set bilangan bulat dan target jumlah, tentukan apakah ada subset yang jumlahnya sama dengan target.
- **Pendekatan Brute-Force:** Memeriksa semua kemungkinan subset untuk menemukan solusi tepat.

Sesi 25: Algoritma Pendekatan - Subset-Sum

1. Konsep Kunci

- **Pemrograman Dinamis:** Peningkat masalah-masalah dasar pemrograman dinamis.
- **Teori Graf:** Peninjauan konsep seperti Komponen Terhubung Kuat (SCC) dan Algoritma Tarjan.
- **NP-Completeness:** Peningkat kelas kompleksitas dan masalah NP-Complete.

2. Teknik Pendekatan Lanjutan

- **Teknik-Teknik Canggih:** Eksplorasi teknik-teknik canggih untuk mengembangkan algoritma pendekatan di luar metode dasar.

Session 26: Tinjauan dan Recap

- **Ringkasan Sesi Sebelumnya:**

- Tinjauan menyeluruh tentang konsep-konsep kunci yang dibahas dalam sesi sebelumnya, termasuk Pemrograman Dinamis, Teori Graf, NP-Completeness, dan Algoritma Aproksimasi.
- Penekanan pada pentingnya meninjau dan memperkuat konsep dasar untuk membangun pemahaman yang solid tentang topik lanjut.
- **Tinjauan Mendalam tentang Pemrograman Dinamis:**
 - **Masalah Kunci:** Tinjauan rinci tentang masalah seperti Painting the Fence dan Longest Palindrome Subsequence.
 - **Teknik:** Strategi untuk mengoptimalkan solusi menggunakan teknik DP, termasuk memoization dan tabulasi.
 - **Contoh Masalah:** Panduan langkah demi langkah untuk menyelesaikan masalah DP kompleks, menyoroti jebakan umum dan praktik terbaik.
- **Teori Graf yang Diperbarui:**
 - **Komponen Terkoneksi Kuat (SCCs):**
 - Penjelasan rinci tentang Algoritma Tarjan dan penerapannya pada analisis graf.
 - Implikasi Praktis: Implikasi praktis dari mengidentifikasi SCCs di berbagai domain, seperti jaringan sosial dan graf web.
 - **Algoritma Graf Lanjut:** Eksplorasi lebih lanjut tentang algoritma graf canggih dan penerapannya.
- **NP-Completeness dan Kelas Kompleksitas:**
 - **Analisis Kompleksitas:**
 - Diskusi mendalam tentang NP-Completeness dari masalah tertentu, termasuk pembuktian teoretis dan implikasi praktis.
 - **Solusi Waktu Polinomial:** Skenario di mana masalah NP-Complete spesifik dapat diselesaikan secara efisien menggunakan algoritma khusus.
 - **Studi Kasus:** Contoh dunia nyata yang menggambarkan aplikasi praktis dan keterbatasan masalah NP-Complete.